

Backlog

Backlog

Vista unificada del trabajo de Equipo 2 (Capacitación + Tutores). Cada épica lista user stories, endpoints, criterio de aceptación y esfuerzo (real para las completadas, estimado para las pendientes).

Estado del modelo 5P y arquitectura: [5p-coverage.md](#), [ai-integration.md](#), [scope-equipos.md](#).

Resumen

	Épica	Estado	Esfuerzo
C1	MVP backend FastAPI + módulo users	✓ Completada	~4 h
C2	Módulo learning_paths con MockPlanGenerator	✓ Completada	~6 h
C3	Persistencia en PostgreSQL (Neon + Render)	✓ Completada	~5 h
C4	Generación con IA: Groq + Gemini con fallback	✓ Completada	~6 h
C5	Módulo attempts con Paciencia (4° P)	✓ Completada	~3 h
C6	Módulo resources con anti-alucinación de URLs	✓ Completada	~3 h
1	Perseverancia y métricas del alumno	⚠ Pendiente	3-4 h
2	Tutores y asignaciones	⚠ Pendiente	4-5 h
3	Sistema de alertas	⚠ Pendiente	3-4 h
4	Next-unit logic	⚠ Pendiente	2-3 h
5	Validaciones críticas	⚠ Pendiente	3 h
6	Auth (mock + JWT real)	⚠ Pendiente	9-12 h
7	Eventos hacia Equipo 1	⚠ Pendiente	2 h
8	Calidad técnica (tests, CORS, Alembic)	⚠ Pendiente	4-6 h

Orden sugerido para cerrar el MVP del modelo

1. Épica 1 — Perseverancia (cierra las 5P).
2. Épica 2 — Tutores (demuestra HITL).

3. Épica 3 — Alertas.
4. Épica 4 — Next-unit logic.

Las cuatro juntas dejan el sistema completo según los documentos del modelo. Total estimado: ~13-16 horas.

Épicas completadas

Épica C1 — MVP backend FastAPI + módulo users

Levantar el esqueleto del backend con el patrón de módulo que se replica en todo el sistema, y entregar el primer módulo de dominio (`users`) para registrar a los actores del modelo.

User stories:

- Como dev quiero un patrón consistente (router/service/repository/models/schemas) para que el código sea mantenible cuando crezca.
- Como admin quiero registrar usuarios con rol (`student` , `tutor` , `company_admin` , `admin`) para identificar a los actores del sistema.
- Como sistema quiero validar que no se repitan emails.

Endpoints entregados:

- `POST /users` , `GET /users` , `GET /users/{id}` , `PATCH /users/{id}` , `DELETE /users/{id}`
- `GET /health`

Criterio de aceptación cumplido:

- Email duplicado devuelve `409 Conflict` .
- ID inexistente devuelve `404` .
- Patrón de módulo documentado en `docs/backend-onboarding.md` .

Commits: `be3462d` (Initial FastAPI MVP backend).

Épica C2 — Módulo `learning_paths` con MockPlanGenerator

Implementar el primer módulo pedagógico que consume el GapReport del Equipo 1 y genera un plan con módulos por skill y unidades por fase 5P (Pasión / Play / Práctica). Sin IA todavía, para validar la estructura y el contrato.

User stories:

- Como sistema quiero recibir un GapReport y producir un plan con módulos ordenados por prioridad.
- Como sistema quiero rechazar GapReports donde todas las skills estén en READY (no hay nada que enseñar).
- Como dev quiero poder cambiar el generador (Mock → IA real) sin tocar el resto del sistema.

Endpoints entregados:

- `POST /learning-paths`
- `GET /learning-paths`
- `GET /learning-paths/{id}`
- `GET /students/{email}/learning-paths`

Criterio de aceptación cumplido:

- Contrato del GapReport alineado con `docs/gap-engine-mvp.md`.
- Patrón Protocol + factory en `plan_generator/` para soportar múltiples implementaciones.
- 3 fases 5P (Pasión / Play / Práctica) por skill MISSING o NEEDS_WORK.
- Cobertura del modelo 5P documentada en `docs/5p-coverage.md`.

Commits: `16873c0`, `2be5e49` (persistence layer Protocol + in-memory).

Épica C3 — Persistencia en PostgreSQL (Neon + Render)

Llevar el backend de in-memory a una base real y deployarlo en internet con auto-deploy desde main.

User stories:

- Como cliente externo quiero que la API esté disponible en una URL pública.
- Como sistema quiero que los datos sobrevivan a reinicios del contenedor.
- Como dev quiero auto-deploy desde main para no manejar deploys manuales.

Cambios entregados:

- Web Service en Render (free tier) con auto-deploy desde main.
- PostgreSQL en Neon (free tier serverless).
- `app/config.py` normaliza la URL al driver `postgresql+psycopg://`.
- `app/main.py` corre `Base.metadata.create_all` en el lifespan startup.
- Migración del repositorio de `learning_paths` a SQLAlchemy con `plan_json` como JSON serializado.
- Pin de Python 3.12 en `.python-version` (sin esto Render usa 3.14 y SQLAlchemy 2.0.36 rompe).

Criterio de aceptación cumplido:

- URL pública: `https://capacity-ar-ap.onrender.com`.
- Push a main dispara redeploy automático en 2-4 min.
- Datos persisten entre reinicios del contenedor.

Commits: `f3580ec`, `6d92bce`, `c67b725`, `700051c`.

Épica C4 — Generación con IA: Groq + Gemini con fallback

Reemplazar el MockPlanGenerator por LLMs reales, manteniendo la estructura híbrida (código determinístico para la estructura del plan, LLM solo para el contenido). Multi-proveedor para no depender de uno solo.

User stories:

- Como alumno quiero contenido educativo personalizado a mis intereses y empresa objetivo, generado en tiempo real.
- Como sistema quiero soportar dos proveedores de IA y elegirlos por env var.
- Como sistema quiero nunca romper la API por culpa del LLM: si falla, caer a Mock automáticamente.
- Como dev quiero pedir ejercicios estructurados (5 multiple_choice con A/B/C/D) que sean evaluables automáticamente.

Cambios entregados:

- `app/shared/llm.py` con `GeminiClient` y `GroqClient` (misma interfaz).
- `plan_generator/_base_llm.py` con clase base abstracta que paraleliza las llamadas con `ThreadPoolExecutor`.
- `GeminiPlanGenerator` y `GroqPlanGenerator` como sub-clases de 4 líneas.
- Factory que lee env var `PLAN_GENERATOR` y envuelve en `_PlanGeneratorWithMockFallback`.
- Prompts versionados en `prompts.py` con instrucciones específicas por fase 5P (tono rioplatense, conexión con intereses, ejercicios A/B/C/D).
- Schema de respuesta del LLM declarado explícitamente para evitar features que Gemini no soporta.

Criterio de aceptación cumplido:

- `PLAN_GENERATOR=groq` genera plan con `generator_used: "groq"` en 5-15s.
- `PLAN_GENERATOR=gemini` genera plan equivalente con `generator_used: "gemini"`.
- Si el LLM falla (cuota, timeout), el plan se devuelve con `generator_used: "mock"`, sin romper.
- 5 ejercicios multiple_choice por skill con `expected_answer` como letra única.
- Cambiar de proveedor en producción no requiere tocar código.

Commits: `7209bb5`, `c2b1e14`, `d5083a5`.

Épica C5 — Módulo attempts con Paciencia (4° P)

Cerrar el ciclo del alumno: que pueda responder ejercicios, recibir feedback empático y avanzar según mastery 80%. Activa la 4° P del modelo 5P de forma transversal.

User stories:

- Como alumno quiero responder un ejercicio del plan y saber si acerté.
- Como alumno quiero recibir un mensaje empático cuando me equivoco, sin que me revelen la respuesta.
- Como sistema quiero calcular el mastery por skill (aciertos / intentos) y marcar cuando supera el 80%.
- Como alumno quiero ver mi historial de intentos.
- Como sistema quiero rechazar intentos sobre planes que no son del alumno (403).

Endpoints entregados:

- `POST /attempts`
- `GET /attempts/{id}`
- `GET /students/{email}/attempts`

Cambios entregados:

- Tabla `attempts` con `learning_path_id` + `module_index` + `unit_index` + `exercise_index` como identificador compuesto (sin tocar el `plan_json`).
- Evaluación por string match (sirve para multiple_choice).
- `attempts/feedback.py` con generación de feedback empático vía LLM activo (Groq o Gemini).
- Fallback a mensaje estático si el LLM falla.

Criterio de aceptación cumplido:

- Respuesta correcta: `is_correct: true`, mensaje de felicitación determinístico con `skill_mastery`.
- Respuesta incorrecta: `is_correct: false`, mensaje empático generado por IA que NO revela la respuesta esperada.
- Intento sobre plan ajeno: `403 Forbidden`.
- Mastery se recalcula al consultar el historial (no se persiste, se computa).

Commits: `6a33951`.

Épica C6 — Módulo resources con anti-alucinación de URLs

Enriquecer cada unit del plan con material de estudio externo (videos, guías, sandboxes). Generado por LLM pero con guardrails para que los links no apunten a recursos inexistentes.

User stories:

- Como alumno quiero ver videos, guías y sandboxes asociados a cada unit del plan.
- Como sistema quiero cachear los recursos por (skill, fase) para no llamar al LLM cada vez.
- Como sistema quiero garantizar que los links generados no sean URLs alucinadas por el LLM.

Cambios entregados:

- Tabla `resources` con clave funcional (`skill_name, phase`).
- `resources/suggester.py` con prompt que pide al LLM `title + source + search_terms` (no URL directa).
- Whitelist de dominios estables (`_KNOWN_DOC_DOMAINS`): MDN, git-scm.com, react.dev, freecodecamp.org, docs.python.org, etc.
- Para videos: armado de URL como `https://www.youtube.com/results?search_query=...` (búsqueda, no link directo).
- Para docs canónicas: URL aceptada solo si matchea la whitelist.
- Pattern cache-aside en `resources/service.py`: si hay rows en BD reutilizar, si no pedir al LLM y persistir.
- Embed de los recursos dentro del `plan_json` al generar el plan.

Criterio de aceptación cumplido:

- Cada unit del plan generado trae 2-3 recursos en `unit.resources[]`.
- Las URLs de videos son links de búsqueda (nunca mueren).
- Las URLs de documentación canónica apuntan a dominios reales whitelisteados.
- Segundo plan con misma skill reutiliza los mismos recursos (cero llamadas al LLM).

Commits: `5c0f901`.

Épicas pendientes

Épica 1 — Perseverancia y métricas del alumno

Cierra la 5° P del modelo (Perseverancia) implementando la visualización del progreso y el tracking de actividad.

Módulos a crear: `sessions` + `student_metrics`.

User stories:

- Como alumno quiero ver mi progreso (% completado, hitos, streak de días) para mantenerme motivado, sin ver mi `readiness_score` numérico crudo.
- Como alumno quiero saber qué skills dominé y cuáles me faltan.
- Como tutor quiero ver métricas detalladas de mis asignados (frecuencia, tasa de éxito, dominios estancados).
- Como sistema quiero registrar las sesiones del alumno (inicio, fin, duración).

Endpoints:

- `POST /sessions/start`
- `POST /sessions/{id}/end`
- `GET /students/{email}/sessions`
- `GET /students/{email}/progress` (vista motivacional para el alumno)
- `GET /students/{email}/metrics` (vista de seguimiento para el tutor)

Criterio de aceptación:

- El alumno ve "Vas por el 40%", "Dominaste 2 de 5 habilidades", "Te faltan 3 ejercicios para dominar SQL", "7 días seguidos aprendiendo".
- El alumno NO ve el `readiness_score` numérico.
- El tutor ve frecuencia de sesiones, duración promedio, días de inactividad, ratio teoría/práctica.

Esfuerzo estimado: 3-4 h.

Épica 2 — Tutores y asignaciones

Implementa el HITL del modelo: padrón de tutores + asignación a alumnos + dashboard.

Módulos a crear: `tutors` + `tutor_assignments`.

User stories:

- Como admin quiero registrar tutores con especialidades (skills que enseñan).
- Como admin quiero asignar un tutor a un alumno cuando se crea su plan.
- Como tutor quiero ver mi dashboard con todos mis alumnos asignados y su estado.
- Como tutor quiero ver el detalle de un alumno (progreso, attempts, recursos).

Endpoints:

- `POST /tutors`, `GET /tutors`, `GET /tutors/{id}`, `PATCH /tutors/{id}`

- `POST /tutor-assignments`
- `GET /tutors/{id}/dashboard` (resumen con todos los asignados)
- `GET /tutors/{id}/students/{student_email}` (detalle de un alumno)

Criterio de aceptación:

- Un tutor solo ve a sus alumnos asignados, nunca a alumnos de otros tutores.
- Un alumno tiene a lo sumo un tutor activo por path.
- El dashboard del tutor muestra alertas activas (cuando esté implementada la Épica 3).

Esfuerzo estimado: 4-5 h.

Épica 3 — Sistema de alertas

Detecta señales de riesgo de abandono o frustración y las muestra al tutor.

Módulo a crear: `alerts`.

Tipos de alerta:

Tipo	Trigger	Nivel
<code>LOW_PERSEVERANCE</code>	5 intentos fallidos seguidos en el mismo ejercicio	1
<code>ABANDON_RISK</code>	72hs sin actividad	2
<code>ABANDON_RISK</code>	7 días sin actividad	3 (escala a admin)
<code>LOST_PASSION</code>	Tiempo promedio de sesión cae >50% en 5 días	1
<code>ANOMALY_PATIENCE</code>	Lenguaje agresivo detectado por NLP (post-MVP)	2

User stories:

- Como sistema quiero generar alertas automáticamente cuando se cumplan los triggers.
- Como tutor quiero ver alertas activas de mis asignados, filtrables por nivel.
- Como tutor quiero marcar una alerta como resuelta tras intervenir.
- Como admin quiero recibir alertas nivel 3.

Endpoints:

- `GET /alerts?tutor_id=X&student_email=Y&resolved=false`
- `GET /alerts/{id}`
- `PATCH /alerts/{id}/resolve`

Implementación: generación síncrona dentro de `POST /attempts` (`LOW_PERSEVERANCE`) y un check de inactividad puede correr al consultar el dashboard del tutor (`ABANDON_RISK`).

Criterio de aceptación:

- El alumno NUNCA ve sus alertas.
- El tutor ve solo alertas de sus asignados.
- Una alerta resuelta queda registrada con `resolved_by_tutor_id`.

Esfuerzo estimado: 3-4 h.

Épica 4 — Next-unit logic y desbloqueo progresivo

Le da al alumno una ruta clara de qué hacer ahora y bloquea el avance hasta dominar.

User stories:

- Como alumno quiero saber qué unidad debo hacer ahora según mi progreso.
- Como sistema quiero bloquear las skills siguientes hasta dominar la actual (mastery $\geq$ 80%).
- Como alumno quiero recibir una explicación alternativa si no alcanzo el dominio tras N intentos en una skill.

Endpoints:

- `GET /learning-paths/{id}/next-unit` (devuelve la próxima unit pendiente según mastery)
- `PATCH /learning-paths/{id}/regenerate-unit` (genera contenido alternativo cuando hay bloqueo)

Criterio de aceptación:

- Si el alumno no terminó la unit de Pasión de Git, `next-unit` la devuelve.
- Si dominó Git al 80%, `next-unit` devuelve la primera unit de la siguiente skill.
- Si falló 5 veces seguidas, `regenerate-unit` produce contenido nuevo con prompt alternativo al LLM.

Esfuerzo estimado: 2-3 h.

Épica 5 — Validaciones críticas (entregables con ojo humano)

Algunos entregables requieren juicio humano (presentaciones, code review de proyectos).

Módulo a crear: `validations`.

User stories:

- Como alumno quiero subir un entregable (link a repo, video, presentación) asociado a una unit.
- Como tutor quiero ver entregables pendientes de mis asignados.
- Como tutor quiero aprobar/rechazar un entregable con feedback escrito.
- Como alumno quiero ver el feedback de mis entregables.

Endpoints:

- `POST /validations` (alumno sube link)
- `GET /tutors/{id}/pending-validations`
- `PATCH /validations/{id}/validate` (tutor aprueba/rechaza con feedback)

Esfuerzo estimado: 3 h.

Épica 6 — Auth (fase mock → JWT real)

Activa la matriz de accesos (`access-matrix.md`) que hoy está documentada pero no aplicada. La épica se entrega en dos fases: primero headers mock para destrabar el resto del sistema, después JWT real cuando se necesite producción.

Fase 6A — Headers mock (MVP)

Permite desarrollar y demostrar permisos por rol sin todavía implementar la criptografía. Cuando llegue Fase 6B, el resto del código no se entera.

User stories:

- Como sistema quiero leer `X-User-Id` y `X-User-Email` y `X-User-Role` de los headers HTTP para identificar al usuario actual.
- Como sistema quiero rechazar endpoints según rol declarado en cada router.
- Como dev quiero un único punto donde se decide quién es el usuario actual, para poder reemplazarlo después por JWT sin tocar los routers.

Cambios:

- Crear módulo `app/modules/auth/` :
- `dependencies.py` — define `get_current_user` , `require_role(roles)` , `require_self_or_admin(...)` .
- `schemas.py` — define `AuthenticatedUser` (id, email, role).
- En `dependencies.get_current_user` : lee headers, busca al user en `users` (verifica que el id existe y el rol coincide), devuelve `AuthenticatedUser` . Si falta header o no existe el user → `401` . Si el rol no matchea con el del header → `401` .
- Decorar endpoints con `Depends(require_role(...))` :
- `POST /tutors` → solo `admin` .
- `GET /tutors/{id}/dashboard` → `tutor` (solo si `id` coincide con el suyo) o `admin` .
- `POST /attempts` → `student` (solo sobre planes propios).
- `GET /alerts` → `tutor` (solo de sus asignados) o `admin` .

Criterio de aceptación:

- `POST /tutors` sin headers → `401 Unauthorized` .
- `POST /tutors` con `X-User-Role=student` → `403 Forbidden` .
- `GET /tutors/{id}/dashboard` con `X-User-Id` distinto al `id` del path → `403` .
- `POST /attempts` con `student_email` distinto al `X-User-Email` → `403` (ya está implementado en el service, falta blindar con el header).
- Toda la matriz de `access-matrix.md` queda aplicable con un solo cambio de dependency.

Esfuerzo estimado: 3-4 h.

Fase 6B — JWT real (post-MVP, antes de presentar)

Reemplaza los headers mock por tokens firmados. El resto del código no cambia: solo el contenido de `get_current_user` .

User stories:

- Como alumno/tutor/admin quiero registrarme con email + password y recibir un access token.

- Como usuario autenticado quiero refrescar mi token sin volver a loguearme.
- Como sistema quiero almacenar las passwords hasheadas, nunca en plaintext.
- Como sistema quiero invalidar tokens (logout) ante un evento de seguridad.

Endpoints nuevos:

- **POST /auth/register** — recibe `email`, `password`, `first_name`, `last_name`, `role`. Crea el user con `password_hash`. Devuelve `access_token` + `refresh_token`.
- **POST /auth/login** — recibe `email` + `password`. Devuelve los dos tokens.
- **POST /auth/refresh** — recibe `refresh_token`. Devuelve nuevo `access_token`.
- **POST /auth/logout** — invalida el refresh token del usuario actual.
- **POST /auth/change-password** — recibe password actual + nueva.

Modelo de datos:

- Agregar columna `password_hash` a la tabla `users`.
- Tabla nueva `refresh_tokens` (id, user_id, token_hash, expires_at, revoked_at).

Decisiones técnicas:

- **Hashing:** `passlib[bcrypt]` o `argon2-cffi`. Default bcrypt.
- **Tokens:** `pyjwt` con HS256. Secret en env var `JWT_SECRET_KEY` (mínimo 32 chars random, rotatable).
- **Lifetimes:** access token 15 min, refresh token 7 días.
- **Storage del refresh token:** se devuelve al cliente, se guarda hasheado en BD para poder revocarlo.
- **Cliente:** envía `Authorization: Bearer <access_token>` en cada request.
- **Compatibilidad con Fase 6A:** durante la migración, `get_current_user` acepta tanto JWT como headers mock (controlado por env var `AUTH_MODE=mock|jwt`).

Cambios en `get_current_user`:

```
def get_current_user(authorization: str = Header(None)) -> AuthenticatedUser:
    if settings.auth_mode == "mock":
        # Fase 6A: lee X-User-Id, X-User-Email, X-User-Role
        ...
    else:
        # Fase 6B: parsea el JWT y valida firma + expiración
        token = authorization.removeprefix("Bearer ").strip()
        payload = jwt.decode(token, settings.jwt_secret_key, algorithms=["HS256"])
        return AuthenticatedUser(...)
```

Criterio de aceptación:

- **POST /auth/register** crea user con `password_hash` (nunca plaintext en BD).
- **POST /auth/login** con credenciales válidas devuelve dos tokens.
- Request sin `Authorization` header → `401`.
- Request con token expirado → `401`.
- Request con token modificado (firma inválida) → `401`.
- **POST /auth/refresh** con refresh token revocado → `401`.
- Cambiar `AUTH_MODE=mock` en Render permite volver al modo legacy sin redeploy.

Variables de entorno nuevas:

Variable	Valores	Propósito
<code>AUTH_MODE</code>	<code>mock</code> (default) / <code>jwt</code>	Cuál mecanismo de auth está activo
<code>JWT_SECRET_KEY</code>	string aleatorio	Secret para firmar tokens. Rotar al deployar producción real
<code>JWT_ACCESS_TTL_MINUTES</code>	int (default 15)	Lifetime del access token
<code>JWT_REFRESH_TTL_DAYS</code>	int (default 7)	Lifetime del refresh token

Riesgos / decisiones pendientes:

- ¿Permitimos auto-registro de cualquier rol o solo `student`? Probable: solo `student`; tutores y `company_admin` los crea el admin.
- ¿Email verification? Post-MVP. Por ahora se asume que el email es válido.
- ¿Rate limiting en login? Post-MVP, vía middleware (slowapi).
- ¿OAuth con Google? Útil porque los alumnos ya tienen cuenta Google (Equipo 1 usa form de Google para onboarding). Sería una **Épica 9 separada** si avanza.

Esfuerzo estimado: 6-8 h.

Por qué dividir en fases

La Fase 6A desbloquea el resto del backlog (Tutores, Alertas, Validaciones todas necesitan saber quién es el usuario actual). Hacer JWT directo desde el día 1 retrasa la entrega del TP.

La separación también prueba que el patrón de `Depends(get_current_user)` es el correcto: si la Fase 6B se hace bien, **ningún router cambia** entre las dos fases.

Épica 7 — Eventos hacia Equipo 1

Implementa el contrato Equipo 2 → Equipo 1 definido en `scope-equipos.md`.

User stories:

- Como sistema quiero emitir un evento append-only cada vez que algo relevante pasa en el ciclo del alumno.
- Como Equipo 1 quiero leer la tabla `events` para calcular métricas globales.

Eventos a emitir:

```
session_started, session_ended
exercise_attempted, exercise_completed
unit_completed, module_completed, path_completed
mastery_achieved (skill llega al 80%)
alert_raised, tutor_intervened
```

Cambios:

- Tabla `events` con `student_email`, `type`, `payload_json`, `created_at`.

- Helper `events.emit(type, payload)` llamado desde los services existentes (`attempts.create_attempt`, `sessions.end`, etc.).

Esfuerzo estimado: 2 h.

Épica 8 — Calidad técnica

User stories transversales que no agregan funcionalidad pero suben calidad del entregable.

Items:

- Tests unitarios y de integración con `pytest` y `httpx`. Cobertura mínima del flujo de happy path en cada módulo.
- CORS habilitado para frontend en otro dominio (`fastapi.middleware.cors.CORSMiddleware`).
- Migración de `Base.metadata.create_all` a Alembic para gestionar cambios de schema sin perder datos.

Esfuerzo estimado: 4-6 h total (cada item ~1-2 h).